第36课 DFS经典题 —— 迷宫探路与四皇后

> 本讲义配合课堂例题讲解使用，代码可直接在 Dev-C++ 中运行验证。

五、课堂例题

例题 1：迷宫方案数

题目

给定一个 `n×m` 的迷宫（`n` 行 `m` 列），0 表示可以走，1 表示墙壁。从左上角 `(0,0)` 出发，走到右下角 `(n-1,m-1)`，每次可以上下左右走一步，不能走重复格子。问一共有多少种不同的走法？

输入：第一行两个整数 `n, m`（2 ≤ n, m ≤ 6），接下来 `n` 行每行 `m` 个整数（0 或 1）。保证起点和终点都是 0。

输出：一个整数，表示方案数。

样例输入：

```
4 4
0 0 1 0
0 0 0 0
1 0 1 0
0 0 0 0
```

样例输出：

```
4
```

讲解思路

1. **读入地图**：用 `maze[n][m]` 存储。
2. **从起点开始 DFS**：当前在 `(x, y)`，尝试上下左右四个方向。
3. **三重检查**：不越界、不是墙、没走过。
4. **到达终点**：方案数 `ans++`。
5. **回溯**：取消访问标记，让其他路径也能经过这个格子。

> 特别提醒孩子：这里一定要回溯（擦掉脚印），因为不同路径可以经过同一个格子，只是同一条路径里不能重复走。

完整代码

```
#include <iostream>
using namespace std;

int n, m;           // 迷宫的行数和列数
int maze[10][10];   // 存地图
int vis[10][10];    // 标记是否走过（红油漆）
int ans = 0;        // 记录方案数

// 方向数组：右、左、下、上
int dx[] = {0, 0, 1, -1};
int dy[] = {1, -1, 0, 0};

void dfs(int x, int y) {
    // 到达终点，方案数 +1
    if (x == n - 1 && y == m - 1) {
        ans++;
        return;
    }

    // 尝试四个方向
    for (int i = 0; i < 4; i++) {
        int nx = x + dx[i];  // 新的行
        int ny = y + dy[i];  // 新的列

        // 三重检查
        if (nx >= 0 && nx < n && ny >= 0 && ny < m   // 1. 不越界
            && maze[nx][ny] == 0                       // 2. 不是墙
            && vis[nx][ny] == 0) {                     // 3. 没走过
            vis[nx][ny] = 1;   // 画上红油漆（标记）
            dfs(nx, ny);       // 继续往前走（递归）
            vis[nx][ny] = 0;   // 擦掉红油漆（回溯）
        }
    }
}

int main() {
    cin >> n >> m;

    // 读入地图
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cin >> maze[i][j];
        }
    }
```

```
    // 从起点 (0,0) 出发
    vis[0][0] = 1;   // 起点先标记为走过
    dfs(0, 0);

    cout << ans << endl;
    return 0;
}
```

代码逐行讲解

- `maze[10][10]`：地图数组，开大一点没关系，比 n、m 大就行。
- `vis[10][10]`：访问标记数组，0 表示没走过，1 表示走过了。
- `dx[], dy[]`：方向数组，四个方向一一对应。循环 `i=0,1,2,3` 就能试遍四个方向。
- `dfs(x, y)` 表示"我现在站在 `(x, y)` 这个格子，接下来怎么走"。
- 到终点 `(n-1, m-1)` 就说明找到了一条路，`ans++`。
- `vis[nx][ny] = 1` 和 `vis[nx][ny] = 0` 是一对，进去前画标记，退出来后擦掉——**这就是回溯的灵魂**。

例题 2：4 皇后问题

题目

在 4×4 的棋盘上放置 4 个皇后，使得任意两个皇后不在同一行、同一列、同一对角线上。输出所有合法方案（每行输出 4 个数字，表示该行皇后所在的列号，列号从 1 开始），最后输出方案总数。

输入：无。

输出：每种方案占一行，4 个数字用空格隔开。最后一行输出方案总数。

样例输出：

```
2 4 1 3
3 1 4 2
2
```

讲解思路

1. **逐行放皇后**：第 0 行放好了再放第 1 行，以此类推。每行只放一个（保证不同行）。
2. **检查冲突**：放 `(row, col)` 之前，和前面已放好的每个皇后比较——
   - 列号相同？冲突！
   - 行差的绝对值等于列差的绝对值？在同一对角线上，冲突！
3. **放满 4 行**：输出方案。
4. **回溯**：这一行试完当前列，就去试下一列（`queen[row]` 会被新值覆盖，天然回溯）。

> 画图要点：在黑板 4×4 棋盘上，手动演示放第0行第0列 第1行从第0列开始逐个试 冲突就跳过 找到合法位置 继续第2行……走不通就回到上一行换列。

完整代码

```
#include <iostream>
#include <cmath>
using namespace std;

int n = 4;          // 4皇后
int queen[10];      // queen[i] 表示第 i 行的皇后放在第几列（从1开始）
int ans = 0;        // 方案数

// 检查在第 row 行第 col 列放皇后，是否和前面的皇后冲突
bool check(int row, int col) {
    for (int i = 0; i < row; i++) {
        // 同列冲突
        if (queen[i] == col) {
            return false;
        }
        // 对角线冲突：行差的绝对值 == 列差的绝对值
        if (abs(i - row) == abs(queen[i] - col)) {
            return false;
        }
    }
    return true;  // 没有冲突，可以放
}

void dfs(int row) {
    // 放满了 n 行，找到一个合法方案
    if (row == n) {
        ans++;
        // 输出方案
        for (int i = 0; i < n; i++) {
            cout << queen[i];
            if (i < n - 1) cout << " ";
        }
        cout << endl;
        return;
    }

    // 第 row 行尝试每一列
    for (int col = 1; col <= n; col++) {
        if (check(row, col)) {   // 检查是否合法
            queen[row] = col;    // 放皇后（标记）
            dfs(row + 1);        // 去放下一行（递归）
            // queen[row] 会在下次循环被覆盖，天然回溯
        }
    }
}
```

```
int main() {
    dfs(0);  // 从第 0 行开始放
    cout << ans << endl;
    return 0;
}
```

代码逐行讲解

- `queen[i]`：记录第 `i` 行皇后放在第几列。因为每行只放一个，所以只需要一维数组！
- `check(row, col)`：和已经放好的第 0 ~ row-1 行的皇后一一比较。
- 同列：`queen[i] == col`。
- 对角线：`abs(i - row) == abs(queen[i] - col)`。画图让孩子数格子，发现对角线上行差和列差确实相等。
- `dfs(row)`：从第 `row` 行开始搜索。如果 `row == n` 说明 4 行都放好了。
- 这里不需要显式写 `queen[row] = 0` 来回溯，因为下一次 `col` 循环会自动用新值覆盖 `queen[row]`。但如果用了额外的标记数组（比如 `used_col[]`），就必须回溯。

附：例题源代码与运行结果

例题 1：Maze Path Count

```
// Given an n*m maze (0=path, 1=wall), count all paths
// from (0,0) to (n-1,m-1) using DFS with backtracking.
#include <iostream>
using namespace std;

int n, m;
int maze[10][10];
int vis[10][10];
int ans = 0;

// Direction arrays: right, left, down, up
int dx[] = {0, 0, 1, -1};
int dy[] = {1, -1, 0, 0};

void dfs(int x, int y) {
    // Reached the destination, count this path
    if (x == n - 1 && y == m - 1) {
        ans++;
        return;
    }

    // Try all 4 directions
    for (int i = 0; i < 4; i++) {
        int nx = x + dx[i];
        int ny = y + dy[i];

        // Triple check: in bounds, not a wall, not visited
        if (nx >= 0 && nx < n && ny >= 0 && ny < m
            && maze[nx][ny] == 0
            && vis[nx][ny] == 0) {
            vis[nx][ny] = 1;   // Mark as visited
            dfs(nx, ny);       // Recurse to next cell
            vis[nx][ny] = 0;   // Backtrack: unmark
        }
    }
}

int main() {
    cin >> n >> m;

    // Read the maze
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cin >> maze[i][j];
        }
    }
```

```
    // Start DFS from (0,0)
    vis[0][0] = 1;  // Mark starting point as visited
    dfs(0, 0);

    cout << ans << endl;
    return 0;
}
```

Sample Input

```
4 4
0 0 1 0
0 0 0 0
1 0 1 0
0 0 0 0
```

Expected Output

```
4
```

例题 2：4-Queens Problem

```
// Place 4 queens on a 4x4 board so that no two queens
// attack each other (same row, column, or diagonal).
// Output all valid arrangements and the total count.
#include <iostream>
#include <cmath>
using namespace std;

int n = 4;
int queen[10];   // queen[i] = column of the queen in row i (1-indexed)
int ans = 0;

// Check if placing a queen at (row, col) conflicts with previous queens
bool check(int row, int col) {
    for (int i = 0; i < row; i++) {
        // Same column conflict
        if (queen[i] == col) {
            return false;
        }
        // Diagonal conflict: |row_diff| == |col_diff|
        if (abs(i - row) == abs(queen[i] - col)) {
            return false;
        }
    }
    return true;
}

void dfs(int row) {
    // All n rows filled: found a valid arrangement
    if (row == n) {
        ans++;
        for (int i = 0; i < n; i++) {
            cout << queen[i];
            if (i < n - 1) cout << " ";
        }
        cout << endl;
        return;
    }

    // Try each column for the current row
    for (int col = 1; col <= n; col++) {
        if (check(row, col)) {
            queen[row] = col;    // Place queen
            dfs(row + 1);        // Recurse to next row
            // queen[row] is overwritten in next iteration (natural backtrack)
        }
    }
```

```
}

int main() {
    dfs(0);
    cout << ans << endl;
    return 0;
}
```

Expected Output

```
2 4 1 3
3 1 4 2
2
```